nC

DOCUMENTACIÓN DEL LENGUAJE

nC Origen y guía de uso

Por qué existe nC, cómo preparar las herramientas y cómo leer un programa desde su fuente hasta la capa de máquina.

Edición de bootstrap V81
24 de octubre de 2026
github.com/Arizt00/nc

01 Por qué nace nC
02 Cómo leer el proyecto
03 Preparar las herramientas
04 El primer programa
05 Lugares y funciones
06 Memoria y servicios
07 Leer la capa de máquina
08 Trabajar durante el bootstrap

Por qué nace nC

Un programa termina ejecutándose sobre una máquina concreta. Entre el código que escribe su autor y esa ejecución aparecen decisiones sobre representación, llamadas, memoria, enlace y servicios del sistema. nC quiere que esas decisiones puedan expresarse y seguirse dentro de un mismo lenguaje.

De ahí procede su organización por niveles. Un programa normal describe funciones, datos y flujo. La raíz `[@]` expresa contratos basales. `[@machine]` hace explícitas instrucciones y transiciones de control. `primitive[@]` describe materialidad de bajo nivel. El propósito es que los pasos entre esas superficies conserven información tipada y permitan comprobar qué significa cada operación.

La soberanía del proyecto se refiere a poder estudiar y construir su propia cadena de herramientas. El compilador escrito en nC y la reconstrucción entre generaciones forman parte de ese propósito. Las semillas, los proveedores del sistema y las herramientas externas deben seguir identificados mientras sean necesarios.

La edición V81 documenta reconstrucciones y ejecución en perfiles x86-64 concretos. El cierre global de Stage3 continúa pendiente. Cada avance se incorpora con su alcance y sus pruebas para poder revisar cuánto de ese propósito ya se ha materializado.

nC resulta de interés para quien quiera trabajar con compiladores, representación de datos, ABI o generación nativa. Durante el bootstrap conviene empezar por programas pequeños, conservar la versión del compilador y leer los límites del destino elegido. La semántica manual de memoria exige que el programa controle sus reservas, direcciones y liberaciones.

Cómo leer el proyecto

El lenguaje se llama **nC**. Su compilador se llama **ncc**. Una fuente normal suele usar la extensión `.n`; los ejemplos de máquina usan también `.nm`, y los de primitivas, `.ncp`.

Hay dos recorridos de herramientas que no deben confundirse. El Stage0 retenido ofrece la CLI histórica de desarrollo. El candidato nativo tiene comandos bajo `compiler` y es el que participa en las reconstrucciones B y C del bootstrap. La ayuda del ejecutable que se está usando es la autoridad para sus opciones.

Elemento	Para qué sirve
Programa `.n`	Funciones, lugares tipados, control de flujo e imports
Raíz `[@]`	Contratos basales y continuidad con las capas materiales
`[@machine]`	Instrucciones, registros, etiquetas y finales tipados
`primitive[@]`	Contratos materiales de bajo nivel
Runtime	Servicios y proveedores que necesita un programa
MCT y backend	Materialización, formatos, ABI y destinos

El recorrido inicial recomendado es leer un programa completo, comprobarlo y consultar su representación intermedia. La ejecución nativa añade la selección de destino y los proveedores correspondientes. Un ejemplo de hardware puede servir para análisis sin ser ejecutable en el ordenador del lector.

Los directorios `examples/00_hello`, `examples/01_basics`, `examples/11_machine` y `examples/12_primitive` agrupan ejemplos por tema. Los nombres de hitos como V81 identifican el trabajo de ingeniería; no son números de una versión estable del lenguaje.

Preparar las herramientas

Para una primera prueba, descarga el kit nativo V81 de las Releases del repositorio. Incluye compilador, proveedor, un ejemplo y hashes SHA-256. Ese recorrido no necesita Rust ni LLVM. El kit tiene una CLI parcial de bootstrap; usa los comandos de su README.

En Windows, desde el directorio extraído:

```powershell
.\ncc.exe self-test
.\ncc.exe compiler emit-executable hello.n `
  --target x86_64-pc-windows-msvc --out hello.exe
.\hello.exe
```

En Linux x86-64, conserva el proveedor junto al compilador. El kit fue probado en Ubuntu 24.04 bajo WSL; sus imports requieren glibc 2.34 o posterior.

```bash
chmod +x ncc
./ncc self-test
./ncc compiler emit-executable hello.n \
  --target x86_64-unknown-linux-gnu --out hello
chmod +x hello
LD_LIBRARY_PATH="$PWD${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}" ./hello
```

La salida esperada es `Hello, nC public beta` y el exit code es 0. Los comandos `check`, `machine` y `--help` de Stage0 no forman parte de la interfaz disponible en este kit.

Construir el Stage0 desde fuentes

La construcción actual del Stage0 necesita una semilla nativa de nC. `cargo build` por sí solo no completa esa preparación: el puente `ncc-stage0-bridge` compila la política de Stage0 escrita en nC y exige `NC_STAGE0_POLICY_COMPILER`.

Para trabajar desde fuentes se necesitan Git, la toolchain Rust declarada en `rust-toolchain.toml`, un archivador compatible y una semilla nativa verificada junto a su proveedor de runtime. Comprueba el SHA-256 del paquete descargado antes de seleccionarlo. Conserva juntos el compilador y las bibliotecas que incluya la distribución.

En Linux o WSL, desde la raíz del repositorio y con la semilla en una ruta propia:

```bash
export NC_STAGE0_POLICY_COMPILER="$HOME/nc-seed/ncc"
export NC_STAGE0_POLICY_ARCHIVER=ar
export CARGO_BUILD_JOBS=1
cargo build --locked -p ncc-cli --bin ncc
```

Si el proveedor Linux no está en el RUNPATH del ejecutable, configura `LD_LIBRARY_PATH` con su directorio para esa sesión. En Windows usa una terminal de desarrollo MSVC y establece las mismas variables con rutas absolutas; el archivador debe aceptar `/OUT:`, como `llvm-lib.exe` o `lib.exe`.

```powershell
$env:NC_STAGE0_POLICY_COMPILER = 'C:\nc-seed\ncc.exe'
$env:NC_STAGE0_POLICY_ARCHIVER = 'llvm-lib.exe'
$env:CARGO_BUILD_JOBS = '1'
cargo build --locked -p ncc-cli --bin ncc
```

Estas rutas son ejemplos de instalación. Si falta la semilla, la construcción debe detenerse; un binario antiguo encontrado en otro directorio no sustituye su verificación. La página de descargas identifica qué paquetes han superado el smoke test de publicación.

El primer programa

Este es el ejemplo conservado en `examples/00_hello/hello.n`:

```nc
use <io:output>;

helloCompacto {
    to main {
        println("Hello, nC public beta");
    }

    fa:
        return true;
}
```

El import selecciona la salida de `io`. `helloCompacto` nombra el contenedor. `to main` expresa la entrada compacta; `fa:` conserva el final del contenedor. Las llaves delimitan el programa y su bloque de ejecución.

Con el Stage0 construido desde fuentes puedes comprobarlo e inspeccionar su IR:

```bash
cargo run --locked -p ncc-cli -- check examples/00_hello/hello.n
cargo run --locked -p ncc-cli -- emit-ir examples/00_hello/hello.n
```

El primer comando debe terminar sin error de análisis. El segundo muestra la representación intermedia; su salida no es el texto que imprime el programa. Para la ruta `run`, el entorno también debe disponer del backend y los proveedores que esa ruta necesite.

```bash
cargo run --locked -p ncc-cli -- run examples/00_hello/hello.n
```

La distribución del candidato nativo usa su propia familia de comandos. Sigue la receta que acompaña al paquete para generar y ejecutar la imagen del destino. Conservar esta distinción evita usar una opción de Stage0 con un compilador que ofrece otra CLI.

Lugares y funciones

Una declaración como `contador >> int <- 0;` crea un lugar tipado e inicializa sus bits. `>>` separa nombre y tipo; `<-` escribe. Un lugar sin inicializador no promete memoria inicializada.

```nc
contador {
    fn:
        public:
            run() -> bool {
                valor >> int <- 0;
                while (valor < 3) {
                    valor <- valor + 1;
                }
                return valor == 3;
            }

    ref main ->> run;

    fa:
        return true;
}
```

En esta forma, `fn:` agrupa funciones, `public:` indica visibilidad y `ref main ->> run;` selecciona la entrada. El nombre `run` no tiene un privilegio implícito. Su tipo de retorno es `bool` y la comparación final produce ese valor.

Las funciones reciben parámetros como `valor >> int`. Los bloques `if`, `else` y `while` expresan control de flujo. El lenguaje conserva el orden de evaluación definido por la fuente; la optimización debe respetarlo.

Conviene aprender con ejemplos que devuelvan un resultado fácil de comprobar. Añade después una condición, otra función o un import. Cuando aparezca un diagnóstico, conserva primero el caso mínimo que lo reproduce.

La sintaxis compacta vigente aparece en el parser y en los ejemplos actuales. Las formas históricas pueden permanecer en documentos antiguos o en herramientas de migración. Que un texto antiguo exista en el repositorio no significa que el compilador actual lo acepte.

Memoria y servicios

La superficie normal de nC tiene semántica manual/raw. La comprobación de tipos no añade liberación automática, ownership ni seguimiento de vida de los punteros. El programador debe respetar las condiciones materiales de cada operación.

Las grafías `T*`, `p<T>` y `ptr<T>` convergen en la representación tipada de punteros. `rd`, `wr` y `off` aparecen en ejemplos de acceso a memoria. Antes de utilizarlas, comprueba el contrato de la API, el tamaño de la reserva, la alineación y cuánto tiempo sigue siendo válida la dirección.

Una función que acepta un puntero no recibe por ello la longitud de su reserva. Una llamada que devuelve memoria tampoco implica que el compilador vaya a liberarla. Los imports de memoria deben leerse junto a los ejemplos de su familia y el proveedor del destino.

El runtime proporciona servicios como salida, archivos, cadenas o interacción con el sistema. Su disponibilidad depende del compilador, el proveedor y el perfil. Una firma reconocida por el frontend no demuestra que su símbolo esté disponible al enlazar.

Para dividir un proyecto, el import compacto de módulos usa una forma como:

```
nc
use <project:worker>;
```

La resolución debe conservar la identidad de la unidad y sus dependencias. Evita copiar funciones entre archivos para eludir un problema de enlace: reduce el caso y comprueba el grafo del proyecto. Los objetos y las cachés bajo `.ncc` son resultados de trabajo, no fuentes que deban copiarse al repositorio público.

Leer la capa de máquina

La capa `[@machine]` forma parte del mismo sistema tipado. Los registros, las etiquetas, las instrucciones y los finales participan en su contrato. El ejemplo `examples/11_machine/checked_add_overflow_machine.n` permite observar una suma seguida de una rama por overflow.

```bash
cargo run --locked -p ncc-cli -- check \
  examples/11_machine/checked_add_overflow_machine.n
cargo run --locked -p ncc-cli -- emit-ir \
  examples/11_machine/checked_add_overflow_machine.n
```

La extensión y la superficie importan. Para un módulo `.nm` con contrato de objeto, usa la ruta de máquina correspondiente:

```bash
cargo run --locked -p ncc-cli -- machine emit-ir \
  examples/11_machine/object_elf64.nm
```

Una instrucción de salto necesita un destino coherente. Un terminador cierra un camino de ejecución; no es un adorno del texto. Un bloque que requiere terminar debe hacerlo según su contrato, y el IR tiene que conservar esa decisión.

Los ejemplos con `halt`, registros privilegiados o instrucciones dependientes de ISA se estudian primero con `check` y la salida intermedia. Para ejecutarlos hay que disponer de un destino compatible y conocer su contexto material. La aceptación sintáctica no autoriza a suponer que cualquier máquina anfitriona pueda ejecutarlos.

`primitive[@]` cubre otro nivel de materialidad. Su relación con máquina, memoria y enlace debe mantenerse explícita. Duplicar una semántica en dos superficies crea resultados difíciles de verificar; por eso la continuidad tipada es una regla central del proyecto.

Trabajar durante el bootstrap

Guarda junto a cada reproducción el comando completo, el sistema, el perfil, la versión o hash del ejecutable, la fuente mínima y el diagnóstico. Si comparas generaciones, conserva también los proveedores. Una diferencia de runtime puede parecer un error del compilador.

Las pruebas positivas muestran qué se acepta y qué resultado produce. Las negativas fijan qué debe rechazarse y con qué diagnóstico. Cuando una modificación cambia la superficie o su significado, parser, AST, semántica, IR, fixtures y documentación deben quedar de acuerdo.

El dossier V81 registra reconstrucciones B2 y C2 con identidad del compilador por anfitrión en Windows y Linux/WSL x86-64. También registra programas ejecutados, contratos de runtime y controles adversos. Esos resultados pertenecen al perfil y a la evidencia descritos allí; no convierten todas las arquitecturas declaradas en destinos cerrados.

Para contribuir, elige un comportamiento pequeño: una explicación confusa, un ejemplo que falla con la versión documentada, un diagnóstico incompleto o una regresión reproducible. Abre un issue con el caso mínimo y explica el resultado esperado. Las propuestas de sintaxis requieren justificar cómo se conservará su significado en las capas siguientes.

Fuentes de esta guía

- `AGENTS.md`, reglas de continuidad y criterio de cierre.
- `docs/syntax.md`, superficie compacta y memoria manual.
- `crates/ncc-stage0-bridge/build.rs`, requisito de semilla.
- `examples/00_hello/hello.n` y `examples/11_machine`.
- `docs/NCC_MODULES_V81_DOSSIER.md` y `docs/NCC_MODULES_V81_REPRO.md`.

La referencia de arquitectura explica el pipeline y la lectura de esa evidencia. Los documentos de instalación y descargas del repositorio contienen las instrucciones operativas actualizadas de cada publicación.